🚀 JUnit 5 + Mockito + Spring Boot Testing (One-Chapter Placement Notes)

Goal: Understand what each testing concept is, when to use it, and how they all fit together.

---

1. What is Testing?

Testing is the process of checking whether your code works as expected.

Instead of manually running the application every time, we write **automated tests**.

Example

```
public int add(int a, int b) {
   return a + b;
}
```

Test:

```
assertEquals(5, add(2, 3));
```

---

2. Types of Testing

Type	Tests
Unit Test	One class/method only
Integration Test	Multiple components together (Service + DB)
System Test	Entire application
Acceptance Test	Checks business requirements

Interview Tip:

- **JUnit + Mockito** → Unit Testing
- **@SpringBootTest** → Integration Testing

---

3. What is JUnit?

JUnit is the **most popular Java testing framework**.

It provides:

- Test execution
- Assertions
- Test lifecycle
- Parameterized tests

---

4. Common JUnit Annotations

Annotation	Purpose
@Test	Marks a test method
@BeforeEach	Runs before every test
@AfterEach	Runs after every test
@BeforeAll	Runs once before all tests
@AfterAll	Runs once after all tests
@ParameterizedTest	Runs same test with multiple inputs

Example:

```
@BeforeEach
void setup() {
   System.out.println("Runs before every test");
}
```

---

5. Assertions

Assertions verify the expected result.

Assertion	Purpose
assertEquals()	Compare values
assertTrue()	Expect true
assertFalse()	Expect false
assertNull()	Object should be null
assertNotNull()	Object should not be null
assertThrows()	Expect an exception

Example:

```
assertEquals(4, 2 + 2);
assertNotNull(user);
assertTrue(list.size() > 0);
```

---

6. Parameterized Tests

Run the same test with multiple inputs.

Example:

```
@ParameterizedTest
@CsvSource({
   "1,2,3",
   "2,3,5"
})
void test(int a,int b,int expected){
   assertEquals(expected,a+b);
}
```

No need to write multiple test methods.

---

7. What is @SpringBootTest?

Starts the **entire Spring Boot application** during testing.

@SpringBootTest

class UserServiceTests {

}

Flow:

JUnit

↓

Spring Boot Starts

↓

Beans Created

↓

Autowired Works

↓

Tests Execute

Use when testing with **real Spring beans and database**.

---

8. What is Mockito?

Mockito is a **mocking framework**.

It creates **fake objects** instead of using real ones.

Without Mockito:

Service

↓

Repository

↓

MongoDB

With Mockito:

Service

↓

Fake Repository

---

9. Why Mockito?

Suppose MongoDB is down.

Without Mockito:

❌ Tests fail.

With Mockito:

✅ Tests still run because the repository is fake.

---

10. Important Mockito Annotations

Annotation	**Purpose**
@Mock	Creates fake object
@InjectMocks	Injects fake objects into class being tested
@Spy	Partial mock
@MockBean *(older Spring Boot)*	Replaces Spring bean with mock
@MockitoBean *(newer Spring versions)*	Spring-managed mock bean

---

@Mock

Creates a fake dependency.

@Mock

UserRepository repository;

Creates:

Fake UserRepository

@InjectMocks

Creates the real object and injects mocked dependencies.

@InjectMocks

UserDetailsServiceImpl service;

Flow:

Fake Repository

↓

Injected

↓

Real Service

---

11. when().thenReturn()

Defines mock behavior.

```
when(repository.findByUserName("Rahul"))
      .thenReturn(user);
```

Meaning:

IF

findByUserName("Rahul")

is called

RETURN

user

---

12. verify()

Checks whether a method was called.

```
verify(repository)
    .findByUserName("Rahul");
```

Useful for verifying interactions.

---

13. Argument Matchers

Instead of exact values:

```
when(repository.findByUserName(anyString()))
```

Common matchers:

- any()
- anyString()
- anyInt()
- eq(value)

---

14. Your Two Test Classes

Class 1

```
@SpringBootTest
class UserServiceTests
```

- Starts Spring Boot.
- Uses real UserRepository.
- Connects to real MongoDB.
- **Integration Test**.

Class 2

@Mock

UserRepository repository;

@InjectMocks

UserDetailsServiceImpl service;

- Doesn't start Spring Boot.
- Doesn't use MongoDB.
- Uses fake repository.
- **Unit Test**.

---

15. Unit Test vs Integration Test

Unit Test	**Integration Test**
Uses Mockito	Uses Spring Boot
Fast	Slower
No database	Real database
Tests one class	Tests multiple components
@Mock	@SpringBootTest

---

16. Common Interview Questions

Why use Mockito?

To test business logic without depending on external systems like databases or APIs.

Difference between @Mock and @InjectMocks?

- @Mock → Creates fake dependency.

- @InjectMocks → Creates real object and injects mocks.

Difference between @Mock and @MockBean?

- @Mock → Pure Mockito, no Spring context.
- @MockBean → Replaces a Spring bean inside the Spring context.

Why use @SpringBootTest?

To load the complete Spring application and test real bean interactions.

What does when().thenReturn() do?

Defines what a mocked method should return when called.

What does verify() do?

Checks whether a mocked method was invoked.

---

17. Testing Flow

Integration Test

JUnit

↓

@SpringBootTest

↓

Spring Boot Starts

↓

Real Beans

↓

Real Repository

↓

MongoDB

Unit Test

JUnit

↓

Mockito

↓

Fake Repository

↓

Service

---

18. Best Practices

- ✅ Use **JUnit** for writing tests.
- ✅ Use **Mockito** to mock dependencies.
- ✅ Use @SpringBootTest only when Spring context is needed.
- ✅ Prefer unit tests for business logic (fast).
- ✅ Keep tests independent and deterministic.

---

19. Common Mistakes

- ❌ Using @SpringBootTest for every test (slow).
- ❌ Forgetting to define mock behavior with when().thenReturn().
- ❌ Assuming @Mock creates a real object.
- ❌ Testing multiple responsibilities in one test method.
- ❌ Not verifying important interactions when needed.

🎯 **Revision Box (5-Minute Review)**

- **JUnit** → Java testing framework.
- **Mockito** → Creates fake objects (mocks).
- **Unit Test** → Tests one class, usually with mocks.
- **Integration Test** → Tests multiple components together.
- @Test → Test method.
- @SpringBootTest → Starts complete Spring Boot application.
- @Mock → Fake dependency.
- @InjectMocks → Injects fake dependencies into real object.
- when(...).thenReturn(...) → Defines mock behavior.
- verify(...) → Verifies method invocation.
- **Assertions** (assertEquals, assertTrue, assertNotNull, etc.) → Validate expected results.

Placement Rule:
JUnit = "How to write tests"
Mockito = "How to fake dependencies"
Spring Boot Test = "How to test with the real Spring application"